A lightweight canonicalization engine in C

William Zhang

Master's Thesis Defence, Polytechnique Montreal, Aug. 14, 2026

Outline

Background

Canonicalization

A lightweight C engine

Results

Conclusions

Convex optimization

$$\begin{array}{ll} \text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq 0, \quad i = 1, \ldots, m \\ & Ax = b \end{array}$$

- variable $x \in \mathbf{R}^n$
- equality constraints are linear and $f_0, \ldots, f_m$ are convex

A concrete example: regularized regression

- we want to fit a model to noisy measurements
- data: a feature matrix $F \in \mathbf{R}^{m \times n}$ and observations $g \in \mathbf{R}^m$ (from experiments, sensors, ...)
- unknown: model coefficients $x \in \mathbf{R}^n$

$$\text{minimize} \quad \|Fx - g\|_2 \; + \; \mu \, \|x\|_2$$

- first term fits the data; second term regularizes ($\mu \geq 0$ trades off fit vs. size)
- μ is a hyperparameter — often swept over many values (motivates fast re-solves)

CVXPY

- a **modeling language** for convex optimization, embedded in Python
- the user describes a problem in natural mathematical form

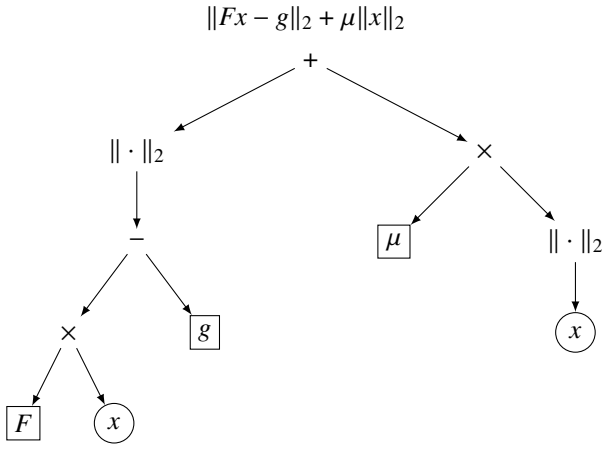
```
import cvxpy as cp

x    = cp.Variable(n)
obj = cp.norm(F @ x - g, 2) + mu * cp.norm(x, 2)
prob = cp.Problem(cp.Minimize(obj))
prob.solve()
```

- `prob.solve()` dispatches to a numerical **solver** (SCS, ECOS, MOSEK, Clarabel, . . .)
- CVXPY is **solver-agnostic** — the same model runs on any of them

Expression trees

CVXPY stores a problem as a tree of **atoms**; leaves are **variables** (circles) and **data/parameters** (boxes).



Standard conic form

$$\begin{array}{ll} \text{minimize} & c^T x + d \\ \text{subject to} & b - Ax \in \mathcal{K} \end{array}$$

- decision variable $x \in \mathbf{R}^n$
- problem data $A \in \mathbf{R}^{m \times n}$, $b \in \mathbf{R}^m$, $c \in \mathbf{R}^n$, $d \in \mathbf{R}$
- $\mathcal{K} \subseteq \mathbf{R}^m$ is a non-empty closed convex cone (*e.g.* $\mathbf{R}^m_+$, second-order, PSD)

Standard conic form

$$\begin{array}{ll} \text{minimize} & c^T x + d \\ \text{subject to} & b - Ax \in \mathcal{K} \end{array}$$

- decision variable $x \in \mathbf{R}^n$
- problem data $A \in \mathbf{R}^{m \times n}$, $b \in \mathbf{R}^m$, $c \in \mathbf{R}^n$, $d \in \mathbf{R}$
- $\mathcal{K} \subseteq \mathbf{R}^m$ is a non-empty closed convex cone (*e.g.* $\mathbf{R}^m_+$, second-order, PSD)
- nearly every convex problem can be cast in this form
- modern conic solvers (SCS, ECOS, MOSEK, Clarabel, ...) all consume this representation

From conic form to problem data

- the conic form is still a set of **affine expression trees** over the variables
- the solver needs **explicit problem data**: matrices and vectors A, b, c (and P for quadratics)
- *matrix stuffing*: walk those trees and emit this data

From conic form to problem data

- the conic form is still a set of **affine expression trees** over the variables
- the solver needs **explicit problem data**: matrices and vectors A, b, c (and P for quadratics)
- *matrix stuffing*: walk those trees and emit this data
- this step has a real cost — especially for **repeated solves** over a family of instances (*e.g.* MPC, hyperparameter sweeps, online learning), where it is paid again and again

From conic form to problem data

- the conic form is still a set of **affine expression trees** over the variables
- the solver needs **explicit problem data**: matrices and vectors A, b, c (and P for quadratics)
- *matrix stuffing*: walk those trees and emit this data
- this step has a real cost — especially for **repeated solves** over a family of instances (*e.g.* MPC, hyperparameter sweeps, online learning), where it is paid again and again
- **this thesis:** we focus on this computational step — converting a set of affine expression trees (the conic form) into explicit problem data — with support for fast **parametric re-solves**

Outline

Background

Canonicalization

A lightweight C engine

Results

Conclusions

Parameters in CVXPY

- **parameterized problems are CVXPY's central use case** — and the central concern of this thesis
- `cp.Parameter` — a symbolic value that changes between solves
- structure of the problem stays fixed; only data values move

```
A = cp.Parameter((m, n))
b = cp.Parameter(m)
lam = cp.Parameter(nonneg=True)
prob = cp.Problem(cp.Minimize(
    cp.norm(A @ x - b, 2) + lam * cp.norm(x, 1)))

for lam_val in np.logspace(-3, 1, 50):
    lam.value = lam_val
    prob.solve()                # <-- want this to be fast
```

- use cases: model-predictive control, LASSO regularization sweeps, online learning

Matrix stuffing

- start simple: **constants and variables only** (no parameters yet)
- after reductions: a tree of affine atoms over variables
- walk the tree, accumulate each leaf's contribution into the right entry of A, b, c
- every nonzero of A corresponds to a (constraint row, variable column) pair touched by the walk

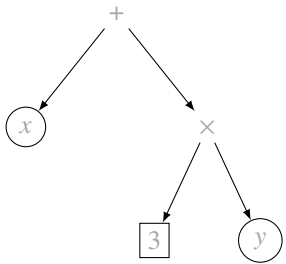
Matrix stuffing

- start simple: **constants and variables only** (no parameters yet)
- after reductions: a tree of affine atoms over variables
- walk the tree, accumulate each leaf's contribution into the right entry of A, b, c
- every nonzero of A corresponds to a (constraint row, variable column) pair touched by the walk
- this is the basic flow — and the current backend re-runs it on every solve

Matrix stuffing: a worked example

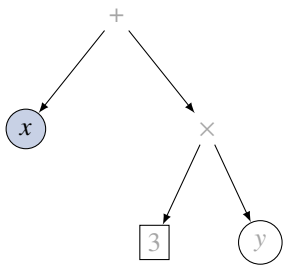
$x + 3y$ with $x, y, z \in \mathbf{R}^2$

row block of A



Matrix stuffing: a worked example

$x + 3y$ with $x, y, z \in \mathbf{R}^2$

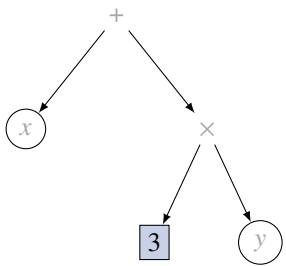


row block of A

- x : $\begin{bmatrix} I & 0 & 0 \end{bmatrix}$

Matrix stuffing: a worked example

$x + 3y$ with $x, y, z \in \mathbf{R}^2$

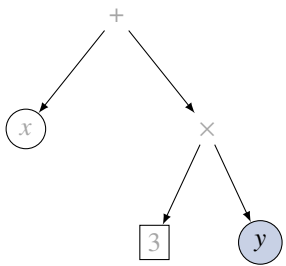


row block of A

- x : $\begin{bmatrix} I & 0 & 0 \end{bmatrix}$
- 3 : $\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$ (constant)

Matrix stuffing: a worked example

$x + 3y$ with $x, y, z \in \mathbf{R}^2$

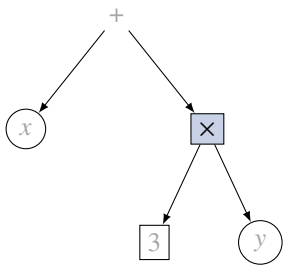


row block of A

- x : $\begin{bmatrix} I & 0 & 0 \end{bmatrix}$
- 3 : $\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$ (constant)
- y : $\begin{bmatrix} 0 & I & 0 \end{bmatrix}$

Matrix stuffing: a worked example

$x + 3y$ with $x, y, z \in \mathbf{R}^2$

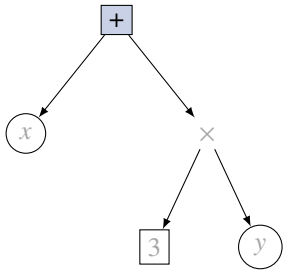


row block of A

- x : $\begin{bmatrix} I & 0 & 0 \end{bmatrix}$
- 3 : $\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$ (constant)
- y : $\begin{bmatrix} 0 & I & 0 \end{bmatrix}$
- $3y$: $\begin{bmatrix} 0 & 3I & 0 \end{bmatrix}$

Matrix stuffing: a worked example

$x + 3y$ with $x, y, z \in \mathbf{R}^2$



row block of A

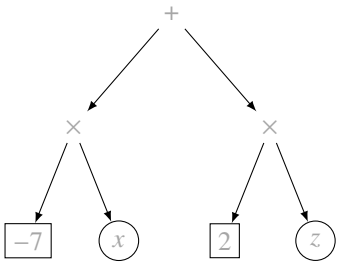
- x : $\begin{bmatrix} I & 0 & 0 \end{bmatrix}$
- 3 : $\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$ (constant)
- y : $\begin{bmatrix} 0 & I & 0 \end{bmatrix}$
- $3y$: $\begin{bmatrix} 0 & 3I & 0 \end{bmatrix}$
- $x{+}3y$: $\begin{bmatrix} I & 3I & 0 \end{bmatrix}$

each block is 2×2; the z block stays 0. This is row block 1 of A.

Stacking constraints

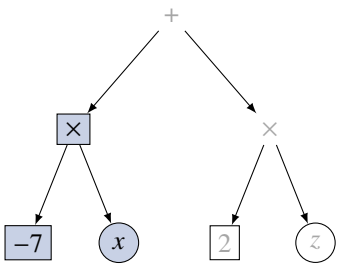
$-7x + 2z$ with $x, y, z \in \mathbf{R}^2$

row block of A



Stacking constraints

$-7x + 2z$ with $x, y, z \in \mathbf{R}^2$

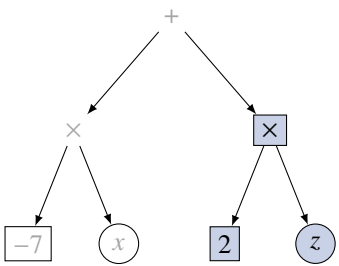


row block of A

- $-7x$: $\begin{bmatrix} -7I & 0 & 0 \end{bmatrix}$

Stacking constraints

$-7x + 2z$ with $x, y, z \in \mathbf{R}^2$

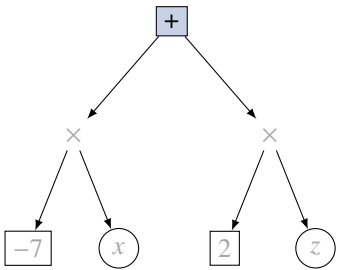


row block of A

- $-7x$: $\begin{bmatrix} -7I & 0 & 0 \end{bmatrix}$
- $2z$: $\begin{bmatrix} 0 & 0 & 2I \end{bmatrix}$

Stacking constraints

$-7x + 2z$ with $x, y, z \in \mathbf{R}^2$

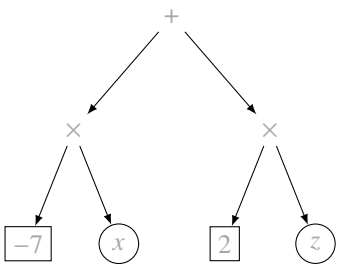


row block of A

- $-7x$: $\begin{bmatrix} -7I & 0 & 0 \end{bmatrix}$
- $2z$: $\begin{bmatrix} 0 & 0 & 2I \end{bmatrix}$
- $-7x+2z$: $\begin{bmatrix} -7I & 0 & 2I \end{bmatrix}$

Stacking constraints

$-7x + 2z$ with $x, y, z \in \mathbf{R}^2$



row block of A

- $-7x$: $\begin{bmatrix} -7I & 0 & 0 \end{bmatrix}$
- $2z$: $\begin{bmatrix} 0 & 0 & 2I \end{bmatrix}$
- $-7x+2z$: $\begin{bmatrix} -7I & 0 & 2I \end{bmatrix}$

stacking both constraints:

$$A = \begin{bmatrix} I & 3I & 0 \\ -7I & 0 & 2I \end{bmatrix} \in \mathbf{R}^{4\times 6}$$

Variable offsets

- the backend stacks all variables into one vector u
- each variable has a unique **id**; `id_to_col` maps it to a **column offset**
- a variable of size n occupies n consecutive columns of A

$$x \in \mathbf{R}^2, \quad y \in \mathbf{R}^3 \qquad \Rightarrow \qquad u = \big[\ \underbrace{x_1\ x_2}_{\text{cols } 1\text{–}2}\ \ \underbrace{y_1\ y_2\ y_3}_{\text{cols } 3\text{–}5}\ \big]^\top \in \mathbf{R}^5$$

$$\texttt{id_to_col} = \{\, \mathrm{id}(x) : 0,\ \ \mathrm{id}(y) : 2 \,\}$$

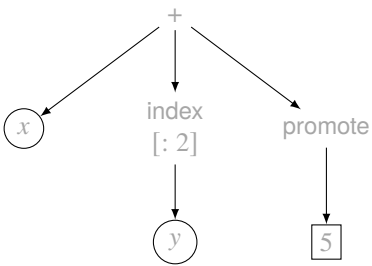
- so A has 5 columns (`var_length`); a final column holds the constant b

Building the coefficient blocks

$$x + y_{[:2]} + 5$$

$$x \in \mathbf{R}^2,\ y \in \mathbf{R}^3$$

each node holds {var id ↦ `scipy.sparse`}

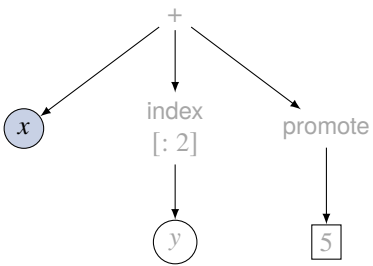


`scipy.sparse`: variable → `eye_array` (CSC); `index` selects rows; `promote` repeats a row; `sum` merges per-variable keys.
Constant id = -1.

Building the coefficient blocks

$$x + y_{[:2]} + 5$$

$x \in \mathbf{R}^2,\ y \in \mathbf{R}^3$



each node holds {var id ↦ `scipy.sparse`}

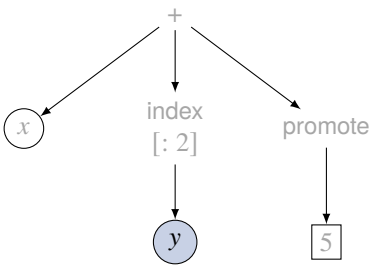
- x : $\{x \mapsto \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]\}$

`scipy.sparse`: variable → `eye_array` (CSC); `index` selects rows; `promote` repeats a row; `sum` merges per-variable keys. Constant id = -1.

Building the coefficient blocks

$x + y_{[:2]} + 5$

$x \in \mathbf{R}^2,\ y \in \mathbf{R}^3$



each node holds {var id ↦ `scipy.sparse`}

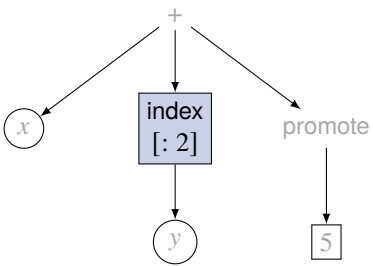
- x : $\{x \mapsto \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]\}$
- y : $\{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{smallmatrix}\right]\}$

`scipy.sparse`: variable → `eye_array` (CSC); `index` selects rows; `promote` repeats a row; `sum` merges per-variable keys. Constant id = -1.

Building the coefficient blocks

$$x + y_{[:2]} + 5$$

$x \in \mathbf{R}^2,\ y \in \mathbf{R}^3$



each node holds {var id ↦ `scipy.sparse`}

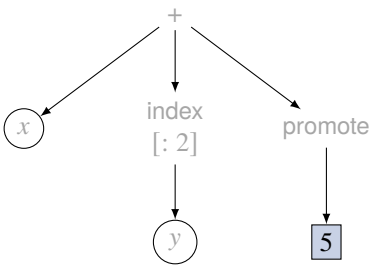
- x : $\{x \mapsto \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]\}$
- y : $\{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{smallmatrix}\right]\}$
- $y_{[:2]}$: $\{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right]\}$ (rows 0, 1)

`scipy.sparse`: variable → `eye_array` (CSC); `index` selects rows; `promote` repeats a row; `sum` merges per-variable keys.
Constant id = -1.

Building the coefficient blocks

$x + y_{[:2]} + 5$

$x \in \mathbf{R}^2,\ y \in \mathbf{R}^3$



each node holds {var id ↦ `scipy.sparse`}

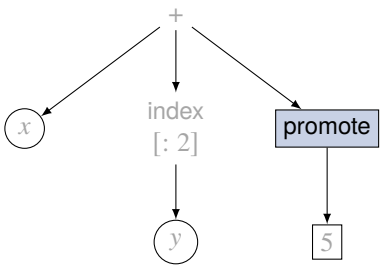
- x : $\{x \mapsto \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]\}$
- y : $\{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{smallmatrix}\right]\}$
- $y_{[:2]}$: $\{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right]\}$ (rows 0, 1)
- 5 : $\{-1 \mapsto [5]\}$

`scipy.sparse`: variable → `eye_array` (CSC); `index` selects rows; `promote` repeats a row; `sum` merges per-variable keys. Constant id = -1.

Building the coefficient blocks

$x + y_{[:2]} + 5$

$x \in \mathbf{R}^2,\ y \in \mathbf{R}^3$



each node holds {var id ↦ `scipy.sparse`}

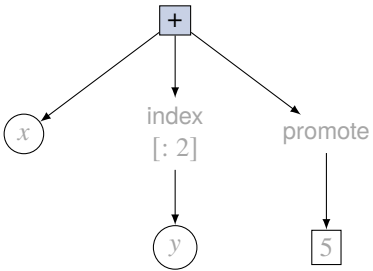
- x : $\{x \mapsto \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\}$
- y : $\{y \mapsto \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\}$
- $y_{[:2]}$: $\{y \mapsto \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}\}$ (rows 0, 1)
- 5 : $\{-1 \mapsto [\,5\,]\}$
- promote: $\{-1 \mapsto \begin{bmatrix} 5 \\ 5 \end{bmatrix}\}$ (repeat row)

`scipy.sparse`: variable → `eye_array` (CSC); `index` selects rows; `promote` repeats a row; `sum` merges per-variable keys.
Constant id = -1.

Building the coefficient blocks

$x + y_{[:2]} + 5$

$x \in \mathbf{R}^2,\ y \in \mathbf{R}^3$



each node holds {var id ↦ `scipy.sparse`}

- x : $\{x \mapsto \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\}$
- y : $\{y \mapsto \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\}$
- $y_{[:2]}$: $\{y \mapsto \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}\}$ (rows 0, 1)
- 5 : $\{-1 \mapsto [5]\}$
- promote: $\{-1 \mapsto \begin{bmatrix} 5 \\ 5 \end{bmatrix}\}$ (repeat row)
- $+$: $\left\{ \begin{matrix} x \mapsto \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},\ y \mapsto \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \\ -1 \mapsto \begin{bmatrix} 5 \\ 5 \end{bmatrix} \end{matrix} \right\}$

`scipy.sparse`: variable → `eye_array` (CSC); `index` selects rows; `promote` repeats a row; `sum` merges per-variable keys. Constant id = -1.

Assembling $[A\ b]$

`id_to_col` places each variable's block at its column offset; the constant fills the last column:

- $\{x \mapsto \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]\} \to$ columns 1–2; $\quad \{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right]\} \to$ columns 3–5

CVXPY: `flatten_tensor` collapses (row, col) into one sparse $[A\ b]$; `id_to_col[-1] = var_length` reserves the constant column.

Assembling $[A\ b]$

`id_to_col` places each variable's block at its column offset; the constant fills the last column:

- $\{x \mapsto \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]\} \to$ columns 1–2; $\quad \{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right]\} \to$ columns 3–5
-

$$[A\ b] = \left[\begin{array}{cc|ccc|c} 1 & 0 & 1 & 0 & 0 & 5 \\ 0 & 1 & 0 & 1 & 0 & 5 \end{array}\right] \in \mathbf{R}^{2\times 6}$$

CVXPY: `flatten_tensor` collapses (row, col) into one sparse $[A\ b]$; `id_to_col`$[-1]$ = `var_length` reserves the constant column.

Assembling $[A\ b]$

`id_to_col` places each variable's block at its column offset; the constant fills the last column:

- $\{x \mapsto \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]\} \to$ columns 1–2; $\quad \{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right]\} \to$ columns 3–5
- $$[A\ b] = \left[\begin{array}{cc|ccc|c} 1 & 0 & 1 & 0 & 0 & 5 \\ 0 & 1 & 0 & 1 & 0 & 5 \end{array}\right] \in \mathbf{R}^{2\times 6}$$
- the y_3 column is all zeros — $y_{[:2]}$ never touched it

CVXPY: `flatten_tensor` collapses (row, col) into one sparse $[A\ b]$; `id_to_col[-1] = var_length` reserves the constant column.

TODO

(to be filled in)

Outline

Background

Canonicalization

A lightweight C engine

Results

Conclusions

The key insight

- for any parameterized problem, under mild conditions the **sparsity pattern** of $A(\theta)$ is fixed across the family
- only the **numerical values** of the nonzeros change with θ

The key insight

- for any parameterized problem, under mild conditions the **sparsity pattern** of $A(\theta)$ is fixed across the family
- only the **numerical values** of the nonzeros change with θ
- reframe matrix stuffing as **sparse Jacobian evaluation**:
  - symbolic phase: compute the sparsity pattern of $\partial A / \partial \theta$ once
  - numeric phase: refill values on every solve
- no restriction on how parameters enter the problem

Two-phase design

Init phase (run once per problem family):

- allocate one `expr` struct per tree node
- traverse the tree to compute sparsity patterns
- allocate Jacobians as CSR matrices

Eval phase (run on every solve, with new θ):

- walk the tree, fill numerical values into pre-allocated buffers
- stuff the resulting A, b, c into the solver

Two-phase design

Init phase (run once per problem family):

- allocate one `expr` struct per tree node
- traverse the tree to compute sparsity patterns
- allocate Jacobians as CSR matrices

Eval phase (run on every solve, with new θ):

- walk the tree, fill numerical values into pre-allocated buffers
- stuff the resulting A, b, c into the solver

Cost model: init is $O(\text{tree size})$ once; eval is $O(\text{nnz}(A))$ per solve.

The two phases on our example

replace the constant 3 with a parameter θ: $\quad x + \theta y, \quad x, y, z \in \mathbf{R}^2$

Init (once per problem family)

- walk the tree, record *which* blocks are nonzero
- pattern: blocks x, y (z empty)
- allocate the CSR matrix $\begin{bmatrix} \bullet & \bullet & 0 \end{bmatrix}$

Eval (every solve, new θ)

- refill values into the fixed pattern
- θ=3 : $\begin{bmatrix} I & 3I & 0 \end{bmatrix}$
- θ=7 : $\begin{bmatrix} I & 7I & 0 \end{bmatrix}$
- structure never recomputed

The two phases on our example

replace the constant 3 with a parameter θ: $\quad x + \theta y, \quad x, y, z \in \mathbf{R}^2$

Init (once per problem family)

- walk the tree, record *which* blocks are nonzero
- pattern: blocks x, y (z empty)
- allocate the CSR matrix $\begin{bmatrix} \bullet & \bullet & 0 \end{bmatrix}$

Eval (every solve, new θ)

- refill values into the fixed pattern
- θ=3 : $\begin{bmatrix} I & 3I & 0 \end{bmatrix}$
- θ=7 : $\begin{bmatrix} I & 7I & 0 \end{bmatrix}$
- structure never recomputed

the current backend redoes the **whole** left column on every solve; the engine pays it once.

(quadratic objectives add a matrix P, built the same way from second-order coefficients.)

The expr struct

```c
typedef struct expr {
    int     d1, d2, size;    /* shape (d1 x d2) */
    int     n_vars;
    int     var_id;          /* offset, or NOT_A_VARIABLE */

    struct expr *left, *right;

    double     *value;       /* forward-pass output */
    CSR_Matrix *jacobian;    /* size x n_vars sparse */

    forward_fn       forward;
    jacobian_init_fn jacobian_init_impl;
    eval_jacobian_fn eval_jacobian;
} expr;
```

The atom contract

Every atom implements three functions:

Function	Phase / Purpose
`forward`	numeric: compute the atom's output value
`jacobian_init`	symbolic: allocate sparsity pattern of the Jacobian
`eval_jacobian`	numeric: fill Jacobian values from children

- `jacobian_init` is called **once** during the init phase
- `eval_jacobian` is called on **every** solve

Atom example: elementwise multiply

- operation: $z = a \odot b$ (elementwise product of two expressions)
- Jacobian: $\partial z/\partial x = \mathrm{diag}(a)\,\partial b/\partial x + \mathrm{diag}(b)\,\partial a/\partial x$

Atom example: elementwise multiply

- operation: $z = a \odot b$ (elementwise product of two expressions)
- Jacobian: $\partial z/\partial x = \operatorname{diag}(a)\,\partial b/\partial x + \operatorname{diag}(b)\,\partial a/\partial x$
- `jacobian_init`: union of the sparsity patterns of $\partial a/\partial x$ and $\partial b/\partial x$
- `eval_jacobian`: scale child Jacobian rows by the current values of a and b

Atom example: elementwise multiply

- operation: $z = a \odot b$ (elementwise product of two expressions)
- Jacobian: $\partial z/\partial x = \mathrm{diag}(a)\,\partial b/\partial x + \mathrm{diag}(b)\,\partial a/\partial x$
- `jacobian_init`: union of the sparsity patterns of $\partial a/\partial x$ and $\partial b/\partial x$
- `eval_jacobian`: scale child Jacobian rows by the current values of a and b
- key observation: the sparsity union is computed **once**; per-solve work is just row scaling
- this is where **parameter values** enter without changing structure

Atom example: matrix multiplication

- operation: $z = A \cdot y$ (where A is a constant or parameter, y is an expression)
- Jacobian: $\partial z / \partial x = A \cdot \partial y / \partial x$

Atom example: matrix multiplication

- operation: $z = A \cdot y$ (where A is a constant or parameter, y is an expression)
- Jacobian: $\partial z / \partial x = A \cdot \partial y / \partial x$
- `jacobian_init`: sparsity pattern of $A \cdot J_y$ derived from the sparsity of A and J_y
- `eval_jacobian`: sparse-sparse multiply, writing into pre-allocated CSR buffers
- the structural pattern — which entries are nonzero — is fixed once A's sparsity is known

Assembling A

- each constraint contributes a row block of A
- after `eval_jacobian` runs on every node, the root Jacobians *are* the rows of A
- we copy them into a flat CSR buffer in the layout the solver expects

Assembling A

- each constraint contributes a row block of A
- after `eval_jacobian` runs on every node, the root Jacobians *are* the rows of A
- we copy them into a flat CSR buffer in the layout the solver expects
- the column structure of A is computed once; row values are refilled per solve
- no Python in the hot path; no allocation in the eval phase

What “lightweight” means

- pure C; no Python in the inner loop
- no symbolic graph mutation between solves
- CSR storage throughout; cache-friendly traversals
- no general-purpose AD framework (no JAX, no Autograd) — we only implement the atoms we need

What “lightweight” means

- pure C; no Python in the inner loop
- no symbolic graph mutation between solves
- CSR storage throughout; cache-friendly traversals
- no general-purpose AD framework (no JAX, no Autograd) — we only implement the atoms we need
- contrast with today’s backend:
  - no 3D parameter tensor in memory
  - sparsity computed once, not on every solve
  - no restriction on how parameters enter

Outline

Background

Canonicalization

A lightweight C engine

Results

Conclusions

Benchmark setup

- hardware: laptop (TODO: spec)
- software: CVXPY pinned (TODO: version)
- baselines:
  - CVXPY DPP path (default)
  - CVXPY `ignore_dpp=True`
  - our C engine
- metrics:
  - cold-start time (first solve)
  - warm-restart time (subsequent solves, new θ)
  - peak memory

Problem suite

- **standard benchmarks**
  - LASSO, portfolio optimization, model-predictive control
  - representative of real-world use cases
- **stress tests**
  - large sparse problems, deep expression trees
  - designed to expose canonicalization overhead
- **least squares**
  - bread-and-butter case; well-understood baseline

Hypotheses (no numbers yet)

- **cold-start time:** comparable to ignore_DPP (no caching benefit on first solve)
- **warm-restart time:** approaches the DPP path's amortized cost, without the DPP rule restriction
- **peak memory:** below DPP, because no 3D parameter tensor is materialized

Hypotheses (no numbers yet)

- **cold-start time:** comparable to ignore_DPP (no caching benefit on first solve)
- **warm-restart time:** approaches the DPP path's amortized cost, without the DPP rule restriction
- **peak memory:** below DPP, because no 3D parameter tensor is materialized
- **break-even:** after ~ 2–3 solves on the same problem family, our engine should beat ignore_DPP

Results

TODO: insert benchmark plots + tables here

- runtime comparison (cold + warm)
- memory comparison
- scaling with problem size

Outline

Background

Canonicalization

A lightweight C engine

Results

Conclusions

Summary

- reframed CVXPY matrix stuffing as **sparse Jacobian evaluation**
- built a lightweight **C engine** that:
  - computes sparsity once per problem family
  - refills values per solve
  - avoids the 3D parameter tensor of the DPP path
- works for parameterized problems **without the DPP rule restriction**

Summary

- reframed CVXPY matrix stuffing as **sparse Jacobian evaluation**
- built a lightweight **C engine** that:
  - computes sparsity once per problem family
  - refills values per solve
  - avoids the 3D parameter tensor of the DPP path
- works for parameterized problems **without the DPP rule restriction**
- (pending) benchmark wins over ignore_DPP, competitive with DPP

Limitations and future work

Limitations

- partial atom coverage — not yet a drop-in replacement for CVXPY
- C library, not yet integrated as a CVXPY backend
- single-threaded; no GPU

Future work

- expand atom library to cover all of CVXPY's standard atoms
- ship as a Python-importable CVXPY backend
- explore non-DPP parameterizations where sparsity is still stable

Thank you

Questions?